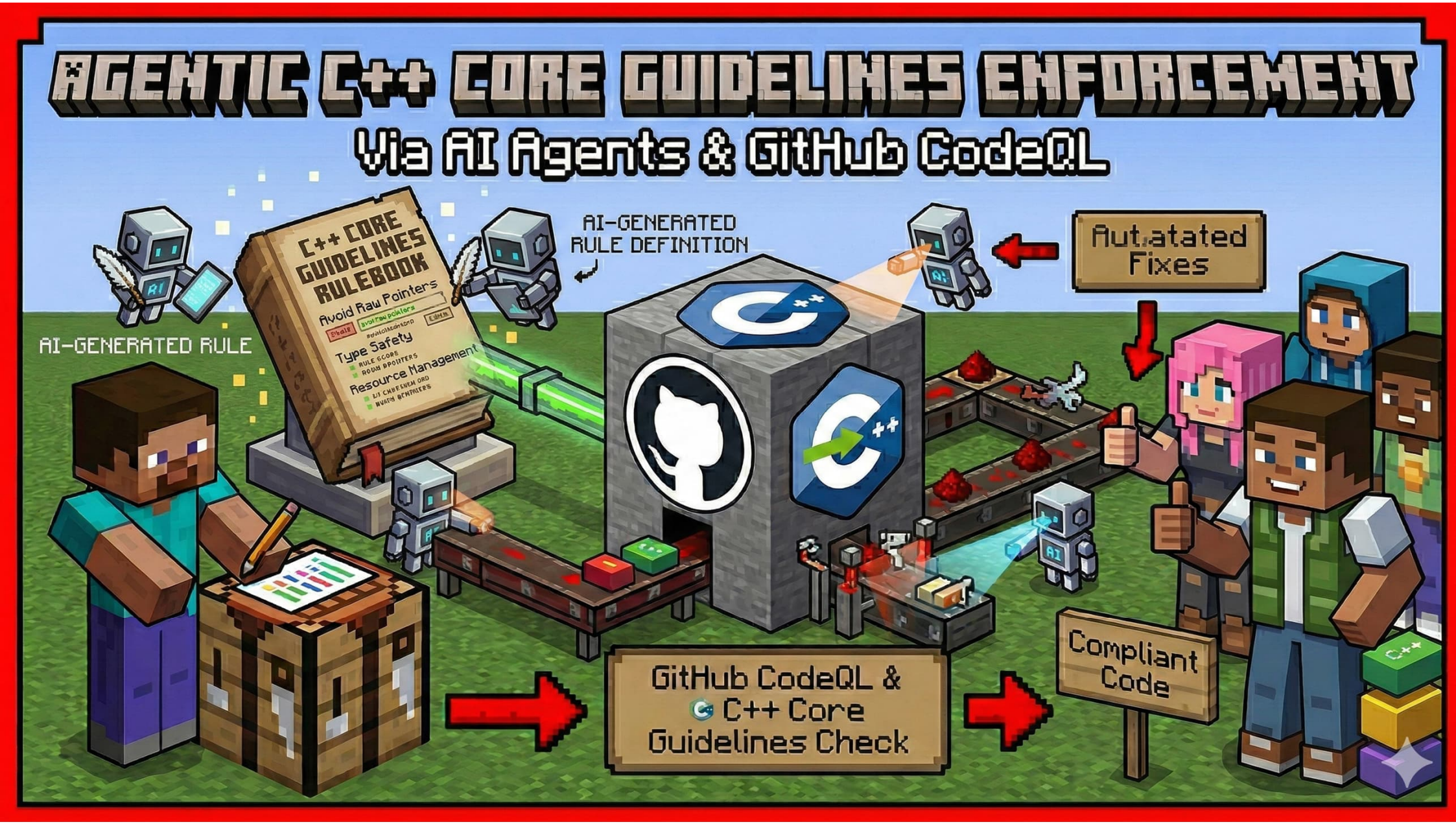

AGENTIC C++ CORE GUIDELINES ENFORCEMENT
Via AI Agents & GitHub CodeQL
C++ CORE GUIDELINES RULEBOOK
Avoid Raw Pointers
Type Safety
Resource Management
AI-GENERATED RULE DEFINITION
AI-GENERATED RULE
Automated Fixes
GitHub CodeQL & C++ Core Guidelines Check
Compliant Code

BERNHARD
MERKLE
github/bmerkle

C++ CORE GUIDELINES
C++ CORE GUIDELINES
SUTTER & STROUSTRUP
C++ CODING STANDARDS
C++ CODING STANDARDS
SUTTER & ALEXANDRESCU
EFFECTIVE C++
EFFECTIVE C++
SCOTT MEYERS
EFFECTIVE JAVA
EFFECTIVE JAVA
JOSHUA BLOCH
EFFECTIVE C++

Turn ON syntax
Top
In: Introduction
P: Philosophy
I: Interfaces
F: Functions
C: Classes and class hierarchies
Enum: Enumerations
R: Resource management
ES: Expressions and statements
Per: Performance
CP: Concurrency
E: Error handling
Con: Constants and immutability
T: Templates and generic programming
CPL: C-style programming
SF: Source files
SL: The Standard library

C++ Core Guidelines

Jul 8, 2025

Editors:

- Bjarne Stroustrup
- Herb Sutter

This is a living document under continuous improvement. Had it been an open-source (code) project, this would have been release 0.8. Copying, use, modification, and creation of derivative works from this project is licensed under an MIT-style license. Contributing to this project requires agreeing to a Contributor License. See the accompanying LICENSE file for details. We make this project available to "friendly users" to use, copy, modify, and derive from, hoping for constructive input.

Comments and suggestions for improvements are most welcome. We plan to modify and extend this document as our understanding improves and the language and the set of available libraries improve. When commenting, please note the introduction that outlines our aims and general approach. The list of contributors is here.

Problems:

- The sets of rules have not been completely checked for completeness, consistency, or enforceability.
- Triple question marks (???) mark known missing information.

AGENTIC AI-DRIVEN C++ CORE GUIDELINES COMPLIANCE FLOW

DATA & KNOWLEDGE INGEST

C++ Core Guidelines (Sutter/Stroustrup)

GITHUB CODEQL DOCUMENTATION

INPUT TO PROMPT

AI AGENT CORE & GENERATION

AI Engine & Agentic Processing

AI DATABASE (STORED DOCS)

GENERATED CODEQL CUSTOM RULE

```
class CodeQQl_Custom Rule
{
  car sputy = {
    soue.appeBn("clsed ");
    conieq.pint=out();
  }
}
```

RULE VALIDATION & TESTING

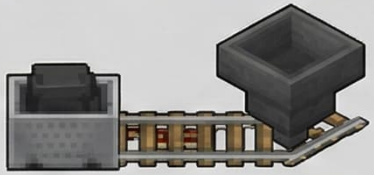
GENERATED CODEQL CUSTOM RULE

```
class CodeQ L Custom Rule
{
  cur aunty = {
    sos.sssgnde("fited ");
    coden.pentnxt(1);
  }
}
```

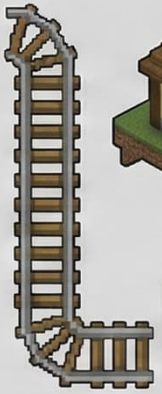
COMPLIANCE TEST POSITIVE

COMPLIANCE TEST NEGATIVE

CI/CD ENFORCEMENT & PROJECT CHECK



GITHUB PIPELINE



CHECKS CODE AGAINST GUIDELINES

```
    exists(MemberVariable mv |
      mv.getEnclosingType() = this and
      (mv.isPrivate() or mv.isProtected())
    )
    or
    exists(Function f |
      f.getEnclosingType() = this and
      (f.isPrivate() or f.isProtected())
    )
    or
    exists(Class c |
      c.getEnclosingType() = this and
      (c.isPrivate() or c.isProtected())
    )
    or
    exists(Struct s |
      s.getEnclosingType() = this and
      s.isPrivate()
    )
    or
    exists(Union u |
      u.getEnclosingType() = this and
      u.isPrivate()
    )
  }
}

from StructWithNonPublicMember s
```

Write a message

Options

Web Search

11 February 2026, 21:57

generate github codeQL to implement the following specified behaviour to check c++ programs:

"Use `class` rather than `struct` if any member is non-public"

please also generate a test-example in C++ code:

first example should follow the rule

second example should violate the rule

11 February 2026, 21:57 / Azure AI GPT 4.1

CodeQL Query: Enforce `class` Instead of `struct` for Non-Public Members

Write a message

Options

Web Search

C++, … its ideas, …its syntax, …

[ ] { } ( ) ;

```
struct Player {
    int id; int score;
    auto operator<=>(const Player&) const = default;
};
```

C++, ... its ideas, ...its syntax, ...

```
// classic Template Meta Programming
template<unsigned int n>
struct Factorial { enum { value = n * Factorial<n - 1>::value }; };

template<> struct Factorial<0> { enum { value = 1 }; };

Factorial<5>::value

// modern constexpr
constexpr int factorial(int n) { return (n <= 1) ? 1 : (n * factorial(n - 1)); }
factorial(5)
```

C++, ... its ideas, ...its syntax, ...

```
// Conecpts
template <typename T>
concept SortableContainer = requires(T c) {
        c.begin(); // Must have a start
        c.end(); // Must have an end
        std::sort(c.begin(), c.end()); // elements inside must be comparable/swappable
};
// Reflection
constexpr auto type_info = ^^Player; // reflection info on Player type
constexpr auto members = std::meta::nonstatic_data_members_of(type_info);
template for (constexpr auto m : members) {
        std::cout << "- " << std::meta::identifier_of(m) << "\n";
}
```

Turn ON syntax
Top
In: Introduction
P: Philosophy
I: Interfaces
F: Functions
C: Classes and class hierarchies
Enum: Enumerations
R: Resource management
ES: Expressions and statements
Per: Performance
CP: Concurrency
E: Error handling
Con: Constants and immutability
T: Templates and generic programming
CPL: C-style programming
SF: Source files
SL: The Standard library

A: Architectural Ideas
NR: Non-Rules and myths
RF: References

C++ Core Guidelines

Jul 8, 2025

Editors:

- Bjarne Stroustrup
- Herb Sutter

This is a living document under continuous improvement. Had it been an open-source (code) project, this would have been release 0.8. Copying, use, modification, and creation of derivative works from this project is licensed under an MIT-style license. Contributing to this project requires agreeing to a Contributor License. See the accompanying LICENSE file for details. We make this project available to "friendly users" to use, copy, modify, and derive from, hoping for constructive input.

Comments and suggestions for improvements are most welcome. We plan to modify and extend this document as our understanding improves and the language and the set of available libraries improve. When commenting, please note the introduction that outlines our aims and general approach. The list of contributors is here.

Problems:

- The sets of rules have not been completely checked for completeness, consistency, or enforceability.
- Triple question marks (???) mark known missing information.
- Update reference sections; many pre-C++11 sources are too old.
- For a more-or-less up-to-date to-do list see: To-do: Unclassified proto-rules.

You can read an explanation of the scope and structure of this Guide or just jump straight in:

- In: Introduction

11 February 2026, 21:53

generate github codeQL to implement the following specified behaviour to check the handling of templates in c++ programs:

"Require a complete set of operations for a concept"

Reason

Ease of comprehension. Improved interoperability. Helps implementers and maintainers.

Example, bad

Copy code

```
template<typename T> concept Subtractable = requires(T a, T b) { a - b; };
```

This makes no semantic sense. You need at least `+` to make `-` meaningful and useful.

Examples of complete sets are

- `Arithmetic` : `+` , `-` , `*` , `/` , `+=` , `-=` , `*=` , `/=`
- `Comparable` : `<` , `>` , `<=` , `>=` , `==` , `!=`

Write a message

Options Web Search

File Edit Selection View Go Run Terminal Help

autogenGitHubCodeQL

EXPLORER

AUTOGENGITHUBCODEQL
- .github
- build
- codeql-custom-queries\cpp
  - C8-use-class-insteadof-struct.ql
  - codeql-pack.lock.yml U
  - long-function.ql
  - qlpack.yml
  - T21-require-complete-set-of-operations-for-con...
- codeql-db
- include
- src
- tests
  - test_C8-use-class-insteadof-struct.cpp
  - test_greeter.cpp
  - test_long-function.cpp
  - test_T21-require-complete-set-of-operations-for...
- videos
- .gitignore
- CMakeLists.txt
- LICENSE
- README.md
- results_t21.sarif
- results.sarif

OUTLINE

TIMELINE

C8-use-class-insteadof-struct.ql | T21-require-complete-set-of-operations-for-concept.ql

codeql-custom-queries > cpp > T21-require-complete-set-of-operations-for-concept.ql > { } T21-require-comp

```
/**
 * @name Incomplete set of overloaded operators
 * @description Flags classes that overload some operators but not a
 *              related set.  For example a class that defines `oper
 *              not `operator==`, or `operator-` but not `operator+`
 *              This is inspired by the T.21 guideline: "Require a c
 *              set of operations for a concept."
 * @kind problem
 * @problem.severity warning
 * @id cpp/incomplete-operator-set
 * @tags maintainability
 *       readability
 */

import cpp

/*
 * Helper: does the class define an operator with the given name?
 * Checks both member operators AND free-standing (non-member) opera
 * whose first parameter type is the class (by value or const-ref).
 * Operator names use the C++ spelling, e.g. "operator<", "operator=
 */
Quick Evaluation: hasOperator
predicate hasOperator(Class c, string op) {
  // Member operator
  exists(MemberFunction mf |
    mf.getDeclaringType() = c and
    mf.getName() = op
  )
  or
  // Free (non-member) operator whose first parameter is the class
  exists(Function f |
    not f instanceof MemberFunction and
    f.getName() = op and
    (
      f.getParameter(0).getType().stripType() = c or
      f.getParameter(1).getType().stripType() = c
    )
  )
}

/*
 * Comparison completeness:
 *   If any ordering operator (<, >, <=, >=) is defined,
```

test_T21-require-complete-set-of-operations-for-concept.cpp

tests > test_T21-require-complete-set-of-operations-for-concept.cpp > Minimal

```
// ────────────────────────────────────────────────────────────────
// Test file for T21 — incomplete operator sets
//
// CodeQL analyses the CLASS DECLARATIONS and their operator
// overloads.  It does NOT need to compile expressions that use
// the missing operators.  The comments below show what would
// fail at compile time if you tried to use the incomplete API.
// ────────────────────────────────────────────────────────────────

//////////////////////////////////////////////////
// BAD example — incomplete operator set
//   (should trigger CodeQL warning)
//////////////////////////////////////////////////

class Minimal {
    int value;
public:
    Minimal(int v) : value(v) {}
    int getValue() const { return value; }
};

// Only operator== and operator< — missing !=, >, <=, >=
bool operator==(const Minimal& a, const Minimal& b) { return a.getVa
bool operator<(const Minimal& a, const Minimal& b)  { return a.getVa

// Only operator+ — missing operator-
Minimal operator+(const Minimal& a, const Minimal& b) { return Minim

// The following calls would FAIL to compile because the operators a
//   if (x != y) { /* ... */ }       // error: no operator!=
//   while (x >= y) { /* ... */ }    // error: no operator>=
//   x += y;                         // error: no operator+=

//////////////////////////////////////////////////
// GOOD example — complete operator set
//   (should NOT trigger CodeQL warning)
//////////////////////////////////////////////////

class Convenient {
    int value;
public:
    Convenient(int v) : value(v) {}
    int getValue() const { return value; }
};
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | AZURE | PLAYWRIGHT

Filter (e.g. text, !excludeText, t... | CMake/Build

```
[cmake] -- Generating done (0.1s)
[cmake] -- Build files have been written to: C:/work/_talks/autogenGitHubCodeQL/build
```

main* | 0 0 | Build | Git Graph | Bernhard Merkle (2 hours ago) | Ln 19, Col 3 | Spaces: 4 | UTF-8 | CRLF | { } C++ | Win32 | Claude Code | CodeQL CLI v2.24.2 | Prettier

bmerkle / autogenGitHubCodeQL

Type / to search

Code | Issues | Pull requests | Agents | Actions | Projects | Wiki | Security 2 | Insights | Settings

Overview

Reporting

Policy

Advisories

Vulnerability alerts

Dependabot

Code scanning 8

Secret scanning

Code scanning

All tools are working as expected

Tools 1 | + Add tool

is:open branch:main

8 Open | 0 Closed | Language | Tool | Rule | Severity | Sort

- **Futile conditional** Note Test — main
  #1 opened 51 minutes ago • Detected by CodeQL in tests/test_T21-require-complet... :60
- **Use `class` rather than `struct` if any member is non-public** Warning Test — main
  #8 opened 33 minutes ago • Detected by CodeQL in tests/test_C8-use-class-instea... :40
- **Use `class` rather than `struct` if any member is non-public** Warning Test — main
  #7 opened 33 minutes ago • Detected by CodeQL in tests/test_C8-use-class-instea... :48
- **Use `class` rather than `struct` if any member is non-public** Warning Test — main
  #6 opened 33 minutes ago • Detected by CodeQL in tests/test_C8-use-class-instea... :56
- **Use `class` rather than `struct` if any member is non-public** Warning Test — main
  #5 opened 33 minutes ago • Detected by CodeQL in tests/test_C8-use-class-instea... :63
- **Use `class` rather than `struct` if any member is non-public** Warning Test — main
  #4 opened 33 minutes ago • Detected by CodeQL in tests/test_C8-use-class-instea... :70
- **Incomplete set of overloaded operators** Warning Test — main
  #3 opened 33 minutes ago • Detected by CodeQL in tests/test_T21-require-complet... :15
- **Example: Find functions longer than 50 lines** Warning Test — main

bmerkle / autogenGitHubCodeQL

Type / to search

Code | Issues | Pull requests | Agents | Actions | Projects | Wiki | Security 2 | Insights | Settings

← Code scanning alerts

Dismiss alert

Use `class` rather than `struct` if any member is non-public #6

Open On branch main • 32 minutes ago

Rule cpp/non-public-struct is not supported by Copilot Autofix for CodeQL

Generate fix

tests/test_C8-use-class-insteadof-struct.cpp:56 Test

```
53  };
54
55  // 7. Struct with a private member function
56  struct BadPrivateFunction {
```

Warning

Use `class` rather than `struct` if any member is non-public

The struct 'BadPrivateFunction' should be declared as a 'class' because it contains non-public members.

CodeQL

```
57      int data;
58  private:
59      void helperFunc() {}
```

Tool	Rule ID
CodeQL	cpp/non-public-struct

Flags struct declarations that have any non-public (private or protected) members.

First detected in commit 32 minutes ago

C8: fixed tests ... ca7ab15

Severity

Warning

Assignees

No one - Assign yourself

Affected branches

main default

First detected 33 minutes ago

Development

Link a branch, pull request, or create a new branch to start working on this alert.

bmerkle / autogenGitHubCodeQL

Type / to search

Code | Issues | Pull requests | Agents | Actions | Projects | Wiki | Security 2 | Insights | Settings

← Code scanning alerts

Dismiss alert

Incomplete set of overloaded operators #3

Open On branch main • 31 minutes ago

Rule cpp/incomplete-operator-set is not supported by Copilot Autofix for CodeQL

Generate fix

tests/test_T21-require-complete-set-of-operations-for-concept.cpp:15 Test

```
12  //   (should trigger CodeQL warning)
13  ////////////////////////////////////////////////
14
15  class Minimal {
```

Warning

Incomplete set of overloaded operators

Class 'Minimal' has an incomplete set of operators: defines operator+ or operator- but not both.
Class 'Minimal' has an incomplete set of operators: defines ordering operators but is missing operator== or operator!=.

CodeQL

```
16      int value;
17  public:
18      Minimal(int v) : value(v) {}
```

Tool	Rule ID
CodeQL	cpp/incomplete-operator-set

Flags classes that overload some operators but not a complete, related set. For example a class that defines `operator<` but not `operator==`, or `operator-` but not `operator+`. This is inspired by the T.21 guideline: "Require a complete set of operations for a concept."

First detected in commit 31 minutes ago

Severity

Warning

Assignees

No one - Assign yourself

Affected branches

main default

First detected 30 minutes ago

Development

Link a branch, pull request, or create a new branch to start working on this alert.

Tags

maintainability readability

Core Guidelines: C++

- Classification
- ✔ automatic enforceable ? → static vs. dynamic
- ✔ not decidable → depending on domain, maybe via metric

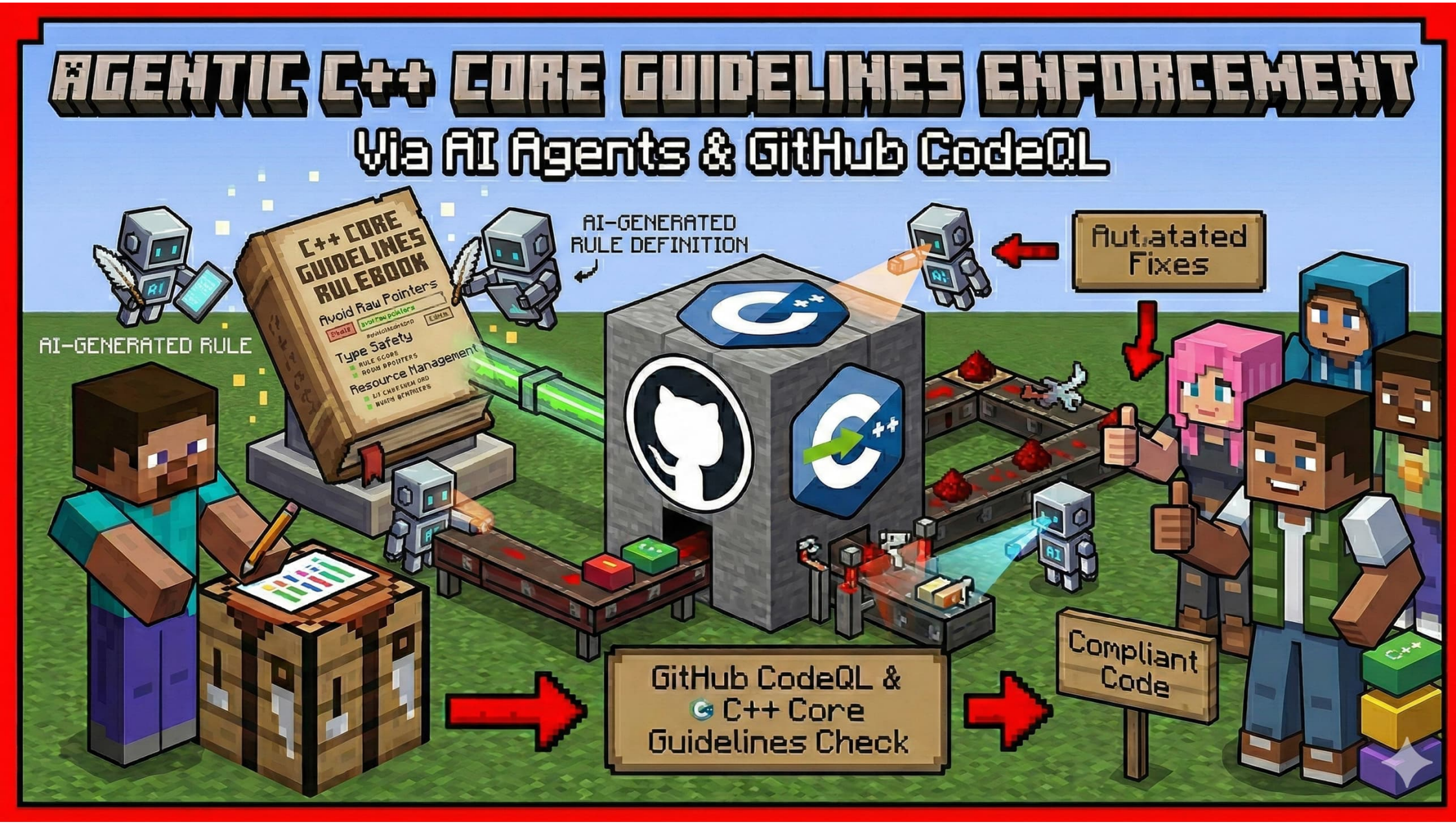
- Examples
- ✔ Prefer `const` and `inline` vs `#define`
- ✔ Use same form in corresponding uses of new and delete
- ✔ Strive for class interfaces that are minimal and complete
- ✔ Choose carefully btw. function overloading / parameter default

Sources

- https://github.com/orgs/codeql/packages
- https://codeql.github.com/
- https://codeql.github.com/codeql-standard-libraries/

- https://github.com/isocpp/CppCoreGuidelines
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

- https://blockbrain.ai/en/


AGENTIC C++ CORE GUIDELINES ENFORCEMENT
Via AI Agents & GitHub CodeQL
C++ CORE GUIDELINES RULEBOOK
Avoid Raw Pointers
Type Safety
Resource Management
AI-GENERATED RULE DEFINITION
AI-GENERATED RULE
Automated Fixes
GitHub CodeQL & C++ Core Guidelines Check
Compliant Code